Open in app ↗

Medium Search Write

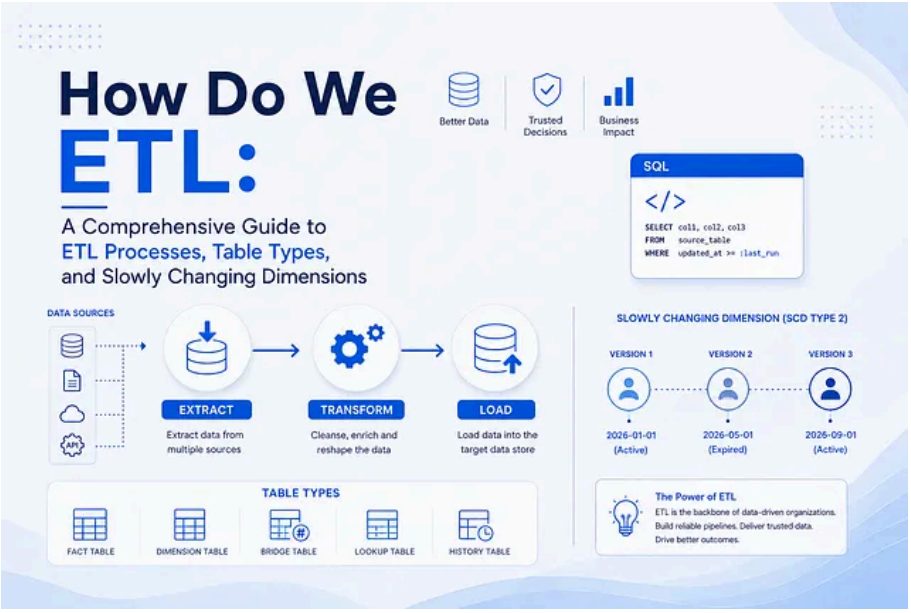
Welcome Offer Access to everything. Now 30% off. Upgrade now

Data Warehouse + Sql + Big Data + Data Analysis + Data Engineering +

How Do We ETL: A Comprehensive Guide to ETL Processes, Table Types, and Slowly Changing Dimensions

Anurag Sharma 17 min read · 3 days ago

4



A practitioner's deep dive into building robust, audit-ready data pipelines — from raw extraction to SCD Type 6 and everything in between.

This comprehensive blog is designed for Data and Analytics Engineers, Architects, and BI Developers who need to cover the end-to-end ETL/ELT lifecycle, the taxonomy of warehouse tables (Facts vs. Dimensions), and a deep dive into Slowly Changing Dimensions (SCD Types 0–6). You will learn implementation patterns for SCD Type 2 using SQL MERGE, performance tuning, and robust migration strategies to evolve your schema without losing history.

The Query That Started a Disaster

It was a routine Monday morning. The sales director walked into the weekly business review and asked a simple question:

> *"How many customers did we have in Q3 of last year, and where were they located?"*

The analyst pulled the number from the data warehouse. The answer was confidently wrong.

Not because the pipeline failed. Not because data was missing. But because when a batch of customers moved cities 14 months ago, the ETL pipeline had overwritten their addresses. No history. No breadcrumbs. Just the present pretending to be the past.

That is the moment most data teams truly understand what Slowly Changing Dimensions are — and why they matter. More importantly, it's the moment they realize ETL is not just about moving data. It's about preserving truth over time.

This guide is the one that analysts wish they had before that Monday morning meeting.

What We'll Cover

This is a long read — intentionally. We're going deep, not just defining terms.

- What ETL actually means in modern data stacks
- The full taxonomy of data warehouse tables
- SCD Types 0 through 6 — with real implementation code
- Migration strategies when you need to change SCD types mid-project
- Practical implementation patterns, tools, and performance tips

Let's start from the beginning.

Part 1: ETL — What It Is and Why It's Still Relevant

The Three Letters Everyone Knows, and Nobody Agrees On

ETL stands for Extract, Transform, Load.

But that three-step definition barely scratches the surface of what modern data engineering actually does.



Extract

Pull data from wherever it lives:

- Relational databases (PostgreSQL, MySQL, Oracle, SQL Server)
- Cloud SaaS platforms (Salesforce, HubSpot, Stripe, Zendesk)
- Flat files (CSV, JSON, Parquet, Avro)
- Event streams (Kafka, Kinesis)
- APIs (REST, GraphQL)
- Application logs

The extract step sounds simple but is frequently the hardest. Sources go offline. APIs rate-limit you. Schema changes without warning. Duplicate records appear. Timestamps are in different time zones.

Transform

This is where the real work happens:

- Clean dirty data (nulls, duplication, encoding issues)
- Validate against business rules
- Standardize formats (dates, currencies, case)
- Enrich with lookups and reference data
- Apply SCD logic (the focus of this guide)
- Aggregate, join, and reshape for the target model

Load

Insert the prepared data into the target system — a data warehouse, lakehouse, or data mart. This includes handling:

- Upserts (update existing + insert new)
- Deduplication
- Partitioning strategy
- Constraint enforcement

ELT: The Modern Variation

In modern cloud warehouses — Snowflake, BigQuery, Databricks, Redshift — the pattern is often ELT: Extract, Load, Transform.

The difference: raw data lands first, then transformations happen inside the warehouse using its native compute.

Approach	Where Transform Happens	Best For
ETL	Before loading (external)	Legacy systems, strict governance
ELT	After loading (inside warehouse)	Cloud-native, scalable, iterative
Reverse ETL	After loading (activated back to ops)	CRM enrichment, personalization

The Core Challenges in ETL

Understanding what makes ETL hard is essential before we build solutions for it.

Challenge	Description
Data Volume & Velocity	Terabytes of daily incremental data, real-time streaming needs
Schema Evolution	Source systems change without notice
Data Quality	Nulls, duplicates, type mismatches, referential integrity failures
Historical Accuracy	How do you answer "what was true on January 15th?"
Idempotency	Running the pipeline twice should not create duplicate records
Latency	Business wants real-time; infrastructure has limits

The historical accuracy challenge is precisely why Slowly Changing Dimensions exist.

Part 2: Types of Tables in a Data Warehouse

Before we can understand SCDs, we need to understand the table ecosystem they live in.

Most enterprise data warehouses follow a Kimball-style dimensional model — or a hybrid of Kimball + Data Vault + One Big Table (OBT) approaches. The key table types are:

1. Fact Tables

The core measurement layer of the warehouse.

- Store numeric, measurable events: sales, clicks, transactions, logins
- Reference dimension tables via surrogate keys

- Usually very large — billions of rows
- Partitioned by date for query performance
- Named with `fact_` or `fct_` prefix

Example: `fact_orders`, `fact_web_sessions`, `fct_revenue`

> *Think of fact tables as the "what happened" tables.*

2. Dimension Tables

The context and description layer.

- Contain descriptive attributes: who, what, where, when, why
- Relatively smaller, but can grow significantly with SCD history
- Subject to SCD logic — this is where change management lives
- Named with `dim_` prefix

Example: `dim_customer`, `dim_product`, `dim_employee`, `dim_date`

> *Think of dimension tables as the "who or what it happened to" tables.*

3. Bridge / Associative Tables

Handle many-to-many relationships between dimensions.

- A customer can belong to multiple segments
- A product can belong to multiple categories
- Bridge tables hold the relationship keys

4. Aggregate / Summary Tables

Pre-computed rollups for performance:

- Materialized views
- Weekly/monthly summary tables
- Mart-level aggregations

Useful when base tables are too large for interactive querying.

5. Staging Tables

Temporary landing zones during ETL:

- Raw data lands here first
- Validation and deduplication happen here

- Not exposed to business consumers
- Truncated and reloaded each run (usually)

6. Reference / Lookup Tables

Small, mostly static data:

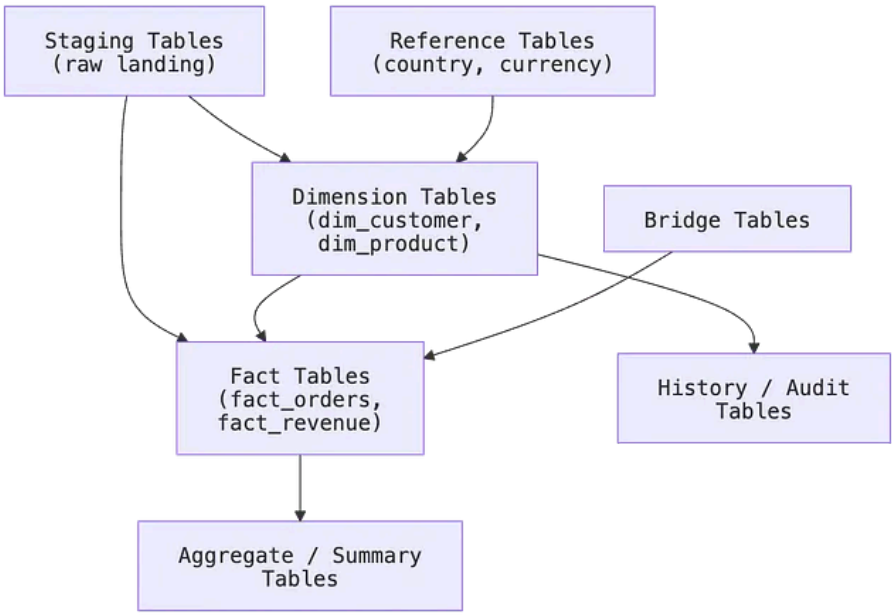
- Country codes
- Currency conversions
- Status codes
- Calendar/holiday tables

7. History / Audit Tables

Separate storage for historical versions:

- Used in SCD Type 4
- Compliance and audit requirements
- Named with `_history` or `_audit` suffix

The Relationship Map



Dimension tables are where the complexity lives — because **business entities change**, and we have to decide what to do when they do.

Part 3: Slowly Changing Dimensions — The Complete Guide

The Core Problem

A customer lives in New York when they sign their first order. Six months later, they move to Chicago. A year after that, they move again to Austin.

Your fact table has three years of purchase orders linked to this customer.

The question: When you run a regional sales report for last year — which city does this customer belong to?

- The city they live in now (Austin)?
- The city they were in when they placed the order (New York or Chicago, depending on the order date)?
- Both, depending on the report?

The answer determines your SCD type.

SCD Type 0: Fixed / Immutable

The philosophy: Some things never change.

Type 0 attributes are permanently locked after their initial load. No updates. No rewrites. No history needed because the value is, by definition, static.

Real examples:

- Customer's original acquisition date
- Product's original manufacturing code
- Employee's hire date
- Financial period start date

Implementation: Enforce at the pipeline level. Reject any incoming updates to these columns. Alternatively, simply never include them in your update logic.

Attribute	Pros	Cons
Complexity	Extremely simple	Inflexible
Storage	Minimal	None lost
History	None needed	None available
Use Case	Truly immutable attributes	Regulatory point-in-time originals

SCD Type 1: Overwrite

The philosophy: History doesn't matter. Only the current truth does.

The simplest approach — and the most dangerous when misapplied. When an attribute changes, you overwrite the old value. No record of what it was before.

Appropriate use cases:

- Correcting a data entry error (misspelled name, wrong email format)
- Updating a field that was always supposed to reflect current state
- Non-analytical descriptive corrections

When it destroys you: Using Type 1 for attributes that business users want to analyze historically — like address, sales region, account manager, or product category — is how you get that Monday morning disaster from our opening story.

Implementation

```
-- SCD Type 1: Simple overwrite
UPDATE dim_customer
SET
    email       = src.email,
    city        = src.city,
    updated_at  = CURRENT_TIMESTAMP
FROM staging_customer src
WHERE dim_customer.customer_id = src.customer_id
  AND (
        dim_customer.email <> src.email
     OR dim_customer.city  <> src.city
  );
```

> *Clean. Simple. And completely unable to answer:* "What city was this customer in when they placed their first order?"

SCD Type 2: Full Versioning (The Industry Standard)

The philosophy: Every change creates a new version of truth.

This is the most powerful and widely used SCD type. Every time a tracked attribute changes, a new row is inserted, preserving the historical record. The old row is closed off. The new row becomes the current record.



This approach requires:

- A surrogate key (system-generated, never business-meaningful) as the primary key
- A natural/business key to identify the entity across versions
- SCD tracking columns to identify the active record and its time boundaries

The Three Pillars of SCD 2 Tracking

These three mechanisms are often used together for maximum robustness:

1. Effective Dates

```
effective_start_date   → when this version became active
effective_end_date     → when this version was superseded (9999-12-31 = still ac
```

2. Version Number

```
version    → 1 for first record, increments with each change
```

3. Active Indicator

```
is_current    → TRUE for the active record, FALSE for historical
```

Full Table Implementation

```
CREATE TABLE dim_customer (
    -- Surrogate Key (system-generated, never exposed to business users)
    customer_sk           BIGINT        PRIMARY KEY,

    -- Natural / Business Key (identifies the entity across all versions)
    customer_id           VARCHAR(50)   NOT NULL,

    -- Tracked Attributes (changes here trigger a new SCD 2 row)
    full_name             VARCHAR(100),
    email                 VARCHAR(150),
    address               VARCHAR(255),
    city                  VARCHAR(100),
    state                 VARCHAR(50),
    country               VARCHAR(50),
    account_tier          VARCHAR(20),

    -- SCD Type 2 Tracking Columns
    effective_start_date  DATE          NOT NULL,
    effective_end_date    DATE          NOT NULL DEFAULT '9999-12-31',
```

```
    version             INT           NOT NULL DEFAULT 1,
    is_current          BOOLEAN       NOT NULL DEFAULT TRUE,

    -- Metadata (optional but highly recommended)
    created_at          TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    updated_at          TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    change_reason       VARCHAR(255),
    source_system       VARCHAR(100)
);
```

ETL Logic for SCD 2

The MERGE pattern is the most efficient way to implement SCD 2 in modern warehouses:

```
-- Step 1: Detect changed records using hash comparison
WITH source_with_hash AS (
    SELECT
        customer_id,
        full_name,
        email,
        city,
        state,
        account_tier,
        MD5(CONCAT_WS('|', full_name, email, city, state, account_tier)) AS row_
    FROM staging_customer
),

current_records AS (
    SELECT
        customer_sk,
        customer_id,
        MD5(CONCAT_WS('|', full_name, email, city, state, account_tier)) AS row_
    FROM dim_customer
    WHERE is_current = TRUE
),

changes AS (
    SELECT s.*
    FROM source_with_hash s
    JOIN current_records c ON s.customer_id = c.customer_id
    WHERE s.row_hash <> c.row_hash  -- Change detected
)

-- Step 2: Expire the old current row
UPDATE dim_customer
SET
    effective_end_date = CURRENT_DATE - 1,
    is_current         = FALSE
FROM changes
WHERE dim_customer.customer_id = changes.customer_id
  AND dim_customer.is_current = TRUE;

-- Step 3: Insert the new version
INSERT INTO dim_customer (
    customer_sk, customer_id, full_name, email,
    city, state, account_tier,
    effective_start_date, effective_end_date, version, is_current
)
SELECT
    NEXTVAL('customer_sk_seq'),
    s.customer_id,
    s.full_name,
    s.email,
    s.city,
    s.state,
    s.account_tier,
```

```
    CURRENT_DATE,
    '9999-12-31',
    COALESCE(d.version, 0) + 1,
    TRUE
FROM source_with_hash s
LEFT JOIN dim_customer d
    ON s.customer_id = d.customer_id
    AND d.is_current = FALSE
    AND d.effective_end_date = CURRENT_DATE - 1
WHERE s.customer_id IN (SELECT customer_id FROM changes);
```

Querying SCD 2 Data

Get the current record:

```
SELECT * FROM dim_customer
WHERE customer_id = 'CUST-1234'
  AND is_current = TRUE;
```

Get the record as of a specific date:

```
SELECT * FROM dim_customer
WHERE customer_id = 'CUST-1234'
  AND '2024-06-15' BETWEEN effective_start_date AND effective_end_date;
```

Join the fact table to the historically correct dimension version:

```
SELECT
    o.order_date,
    o.order_total,
    c.city          AS city_at_time_of_order,
    c.account_tier  AS tier_at_time_of_order
FROM fact_orders o
JOIN dim_customer c
    ON o.customer_sk = c.customer_sk;  -- Surrogate key ensures correct version
```

This last query is why surrogate keys are sacred in SCD 2. The fact table stores the customer_sk at the time of the event — locking in the correct dimension version forever.

SCD 2 Implementation Variants

Variant	Tracking Mechanism	Best For
Dates Only	start_date + end_date	Simple time-range queries
Version Only	version number	Sequential version retrieval
Flag Only	is_current boolean	Simple current-state queries
Hybrid (Recommended)	dates + version + flag	Full flexibility, best query performance
Mini-Dimension	Rapidly changing attributes split into separate dimension	High-velocity attributes (e.g., customer score, segment)

SCD Type 3: Limited History via Additional Columns

The philosophy: Remember the last state, but nothing more.

Instead of new rows, Type 3 adds new columns to store the previous value of a tracked attribute.

```
ALTER TABLE dim_customer ADD COLUMN previous_city        VARCHAR(100);
ALTER TABLE dim_customer ADD COLUMN previous_city_date   DATE;
ALTER TABLE dim_customer ADD COLUMN previous_account_tier VARCHAR(20);
```

customer_id	current_city	previous_city	city_change_date	current_tier	previous_tier
CUST-1234	Chicago	New York	2024-03-01	Gold	Silver

When to Use It

- You need to compare current vs. previous state in reports
- The attribute changes rarely
- Only one step of history is needed
- Schema flexibility is not a constraint

When to Avoid It

- More than one or two levels of history are needed (schema explodes)
- Changes are frequent
- You need genuine time-series analysis

Type 3 is often a stopgap — a “we’ll fix this properly later” solution that tends to stay in production indefinitely.

SCD Type 4: History in a Separate Table

The philosophy: Keep current data fast and clean; archive history separately.

The main dimension table behaves like Type 1 — only the current state exists. All historical versions live in a companion history table.

```
-- Main table: current state only (fast for operational queries)
CREATE TABLE dim_customer (
    customer_sk    BIGINT PRIMARY KEY,
    customer_id    VARCHAR(50),
    full_name      VARCHAR(100),
    city           VARCHAR(100),
    account_tier   VARCHAR(20),
    updated_at     TIMESTAMP
);

-- History table: all versions (used for audit and historical analysis)
CREATE TABLE dim_customer_history (
    history_sk             BIGINT PRIMARY KEY,
    customer_id            VARCHAR(50),
    full_name              VARCHAR(100),
    city                   VARCHAR(100),
    account_tier           VARCHAR(20),
    effective_start_date   DATE,
    effective_end_date     DATE,
    change_reason          VARCHAR(255),
    archived_at            TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

TL Logic

When a change is detected:

1. Copy the existing current row to `dim_customer_history` before overwriting
2. Update `dim_customer` with the new values
3. Record the archival timestamp and change reason

When Type 4 Shines

- History volume is extremely high and would make the main table unmanageable
- Current-state queries are extremely frequent and must be ultra-fast
- Compliance requires historical tracking, but analysts rarely query it
- The history table can live in a separate, cheaper storage tier

SCD Type 5: Type 4 + Mini-Dimension Embedded

A less commonly implemented hybrid that embeds the current mini-dimension key into the fact table alongside the historical Type 4 table.

It's specialized for scenarios where you need both:

- Rapid access to current attribute groups (via mini-dimension FK on the fact table)

- Full historical tracking (via the separate history table)

It's niche — but when the use case fits, it's elegant.

SCD Type 6: The Hybrid King (1 + 2 + 3 Combined)

The philosophy: Give analysts every possible way to slice the data.

Type 6 is the most flexible — and most complex — SCD implementation. It combines:

- Type 1: The current value is always up-to-date on every row
- Type 2: A new row is created for every change
- Type 3: The previous value is stored as an additional column

```
CREATE TABLE dim_customer_type6 (
    customer_sk             BIGINT PRIMARY KEY,
    customer_id             VARCHAR(50),
    full_name               VARCHAR(100),

    -- Type 1: Always reflects the current value (overwritten on all rows)
    current_city            VARCHAR(100),
    current_account_tier    VARCHAR(20),

    -- Type 2: Historical value at the time this version was active
    city_at_version         VARCHAR(100),
    account_tier_at_version VARCHAR(20),

    -- Type 3: Previous value from the prior version only
    previous_city           VARCHAR(100),
    previous_account_tier   VARCHAR(20),

    -- Type 2 tracking columns
    effective_start_date    DATE,
    effective_end_date      DATE DEFAULT '9999-12-31',
    version                 INT,
    is_current              BOOLEAN
);
```

What this enables:

Query	Which Column
"Where does this customer live now?"	`current_city` (Type 1 — same on all rows)
"Where were they when they placed this order?"	`city_at_version` via surrogate key join (Type 2)
"Where did they live before that?"	`previous_city` (Type 3)

SCD Comparison Matrix

SCD Type	History Preserved	New Rows on Change	Schema Change	Complexity	Best For
Type 0	None (immutable)	Never	No	Minimal	Fixed attributes
Type 1	None (overwritten)	No	No	Low	Data corrections
Type 2	Full	Yes	No (just new rows)	Medium	Most analytics use cases
Type 3	One prior version	No	Yes (new columns)	Low-Medium	Simple prev/current comparison
Type 4	Full (separate table)	No (in main)	No	Medium	High-query-volume current state
Type 5	Full (hybrid)	Yes	Yes	High	Mini-dimension analytics
Type 6	Full + Previous + Current	Yes	Yes	High	Full flexibility requirements

Part 4: Migrating Between SCD Types

Changing your SCD strategy mid-project is one of the most delicate operations in data engineering. Requirements evolve. Business priorities shift. What started as "we just need the current state" becomes "we need full history going back to 2019."

Here's how to approach each common migration path.

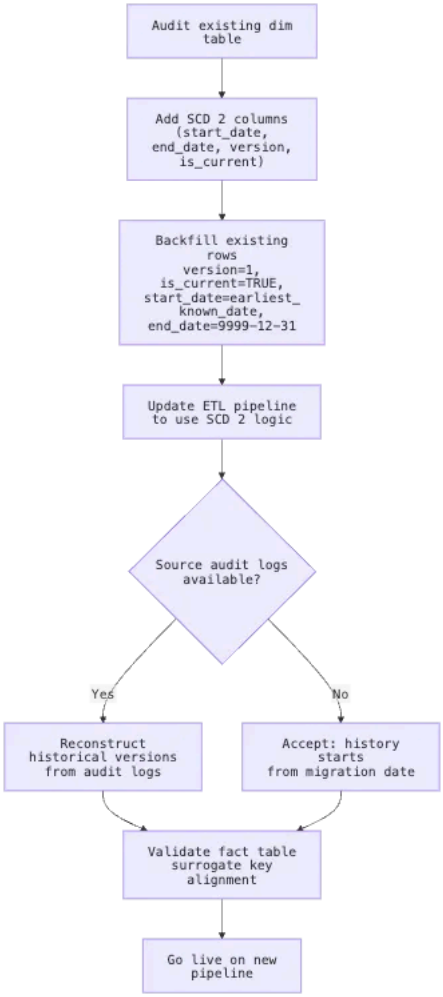
Migration 1: SCD 0 / Type 1 → SCD Type 2

The most common upgrade. You were naïve, and now you need history.

The Challenge

Your dimension has been overwriting values for years. Historical accuracy before the migration date is gone — unless you have source system audit logs.

Migration Steps



SQL Backfill

```sql
-- Step 1: Add SCD 2 columns to existing table
ALTER TABLE dim_customer
    ADD COLUMN effective_start_date DATE,
    ADD COLUMN effective_end_date   DATE DEFAULT '9999-12-31',
    ADD COLUMN version              INT  DEFAULT 1,
    ADD COLUMN is_current           BOOLEAN DEFAULT TRUE;

-- Step 2: Backfill existing records
UPDATE dim_customer SET
    effective_start_date = COALESCE(created_at::DATE, '2015-01-01'),
    effective_end_date   = '9999-12-31',
    version              = 1,
    is_current           = TRUE;

-- Step 3: Add NOT NULL constraints after backfill
ALTER TABLE dim_customer
    ALTER COLUMN effective_start_date SET NOT NULL,
    ALTER COLUMN version SET NOT NULL,
    ALTER COLUMN is_current SET NOT NULL;
```

The Safer Alternative: Shadow Table Migration

Instead of altering the existing table in place (risky), build a new table and cut over:

```
-- Create new SCD 2 table
CREATE TABLE dim_customer_v2 AS
SELECT
    customer_sk,
    customer_id,
    full_name,
    email,
    city,
    COALESCE(created_at::DATE, '2015-01-01') AS effective_start_date,
    '9999-12-31'::DATE                       AS effective_end_date,
    1                                        AS version,
    TRUE                                     AS is_current
FROM dim_customer;

-- Run both pipelines in parallel for 2-4 weeks
-- Cut over via view rename
CREATE OR REPLACE VIEW dim_customer_current AS
SELECT * FROM dim_customer_v2 WHERE is_current = TRUE;
```

Migration 2: SCD Type 2 → SCD Type 1

Rare, but happens when teams want to simplify or reduce storage.

```
-- Step 1: Create simplified table with only current records
CREATE TABLE dim_customer_simplified AS
SELECT
    customer_sk,
    customer_id,
    full_name,
    email,
    city,
    account_tier
FROM dim_customer
WHERE is_current = TRUE;

-- Step 2: Archive the full SCD 2 table (never delete it)
ALTER TABLE dim_customer RENAME TO dim_customer_archive_scd2;

-- Step 3: Rename simplified table
ALTER TABLE dim_customer_simplified RENAME TO dim_customer;
```

> **Important:** Update all fact table references. The surrogate key values may change in the simplified table — validate thoroughly before decommissioning the old table.

Migration 3: SCD Type 2 → SCD Type 3

When you want to simplify querying but need one step of history.

```
-- Add previous value columns to the SCD 2 table
ALTER TABLE dim_customer
    ADD COLUMN previous_city         VARCHAR(100),
    ADD COLUMN previous_account_tier VARCHAR(20);

-- Backfill from historical rows
WITH ranked AS (
    SELECT
```

```
        customer_id,
        city,
        account_tier,
        version,
        LAG(city)         OVER (PARTITION BY customer_id ORDER BY version) AS pr
        LAG(account_tier) OVER (PARTITION BY customer_id ORDER BY version) AS pr
    FROM dim_customer
)
UPDATE dim_customer d
SET
    previous_city         = r.prev_city,
    previous_account_tier = r.prev_tier
FROM ranked r
WHERE d.customer_id = r.customer_id
  AND d.version     = r.version;
```

Migration 4: SCD Type 3 → SCD Type 2

More complex — expanding column-based history into rows.

```
-- Reconstruct version 1 (the "previous" value)
INSERT INTO dim_customer_scd2 (
    customer_sk, customer_id, city, account_tier,
    effective_start_date, effective_end_date, version, is_current
)
SELECT
    NEXTVAL('customer_sk_seq'),
    customer_id,
    previous_city,        -- The Type 3 "previous" column becomes a historical r
    previous_account_tier,
    '2000-01-01',         -- Approximate start date (Type 3 loses this informati
    address_change_date - 1,
    1,
    FALSE
FROM dim_customer
WHERE previous_city IS NOT NULL;

-- Insert the current records as version 2
INSERT INTO dim_customer_scd2 (
    customer_sk, customer_id, city, account_tier,
    effective_start_date, effective_end_date, version, is_current
)
SELECT
    customer_sk,
    customer_id,
    city,
    account_tier,
    COALESCE(address_change_date, created_at::DATE),
    '9999-12-31',
    2,
    TRUE
FROM dim_customer;
```

> ⚠️ Warning: Type 3 → Type 2 migration loses precision. The start date of version 1 must be approximated since Type 3 never stored it. Document this limitation clearly for downstream consumers.

General Migration Best Practices

These principles apply regardless of which SCD migration you're performing:

1. Never delete historical data during migration — Archive, don't truncate. You will need it.
2. Run dual pipelines — Load both old and new SCD table in parallel for 2–4 weeks before cutting over.
3. Use views as the cutover mechanism — Rename views, not tables. Rollback is instant.
4. Validate with as-of queries — Run identical "as-of date" queries against both old and new tables and compare results.
5. Communicate to downstream consumers — Reports, dashboards, ML models that join dimension tables must be retested.
6. Never use natural keys in fact tables — If you've done this, fix it before any SCD migration. It's the root cause of most surrogate key disasters.
7. Document the migration date — Any historical analysis crossing the migration date must account for the data lineage change.

Part 5: Practical ETL Implementation Patterns

Change Detection: The Foundation of Incremental Loads

How do you know what's changed since the last load? Three main approaches:

1. Watermark / High-Water Mark

```
-- Load only records updated since the last successful run
SELECT *
FROM source_customer
WHERE updated_at > (
    SELECT MAX(last_load_timestamp)
    FROM etl_watermark
    WHERE table_name = 'dim_customer'
);
```

Simple and effective. Requires a reliable `updated_at` column on source tables — which many legacy systems don't have.

2. Hash-Based Change Detection

```
-- Compare incoming hash against stored hash
WITH source_hashed AS (
    SELECT
        customer_id,
```

```
        full_name, city, account_tier,
        MD5(CONCAT_WS('|', full_name, city, account_tier)) AS incoming_hash
    FROM staging_customer
),
current_hashed AS (
    SELECT
        customer_id,
        MD5(CONCAT_WS('|', full_name, city, account_tier)) AS stored_hash
    FROM dim_customer
    WHERE is_current = TRUE
)
SELECT s.*
FROM source_hashed s
JOIN current_hashed c ON s.customer_id = c.customer_id
WHERE s.incoming_hash <> c.stored_hash;
```

Robust and portable. Works even without `updated_at` columns. The hash comparison catches any change regardless of which column changed.

CDC (Change Data Capture)

Tools like Debezium, Fivetran, and AWS DMS capture database log events (inserts, updates, deletes) in real time from the source system's transaction log.

Method	Latency	Reliability	Source Requirement
Watermark	Batch	Medium	`updated_at` column
Hash comparison	Batch	High	None
CDC	Near real-time	High	Log access (binlog/WAL)

Loading Patterns

MERGE (Upsert) — The SCD Workhorse

```
MERGE INTO dim_customer AS target
USING staging_customer AS source
    ON target.customer_id = source.customer_id
       AND target.is_current = TRUE

-- No change: do nothing
WHEN MATCHED AND MD5(...) = MD5(...) THEN
    -- nothing

-- Change detected: handled by pre/post logic
WHEN NOT MATCHED THEN
    INSERT (customer_sk, customer_id, full_name, city, ...)
    VALUES (NEXTVAL(...), source.customer_id, source.full_name, source.city, ...
```

> Note: True SCD Type 2 MERGE requires a two-step approach — first expire the old row (UPDATE), then insert the new row (INSERT). Some warehouses support this natively; others require two separate statements.

Querying Historical Data: The As-Of Pattern

```sql
-- What did the customer dimension look like on a specific date?
SELECT
    c.full_name,
    c.city,
    c.account_tier
FROM dim_customer c
WHERE c.customer_id = 'CUST-1234'
  AND '2024-01-15' BETWEEN c.effective_start_date AND c.effective_end_date;

-- Trend analysis: What tier was each customer in, at the time of each order?
SELECT
    o.order_date,
    o.order_total,
    c.city          AS city_at_order_time,
    c.account_tier  AS tier_at_order_time,
    c.version       AS dimension_version
FROM fact_orders o
JOIN dim_customer c ON o.customer_sk = c.customer_sk
ORDER BY o.order_date;
```

The surrogate key join is what makes this possible. Each fact row stores the `customer_sk` that was active at the time the event occurred. The dimension does the rest.

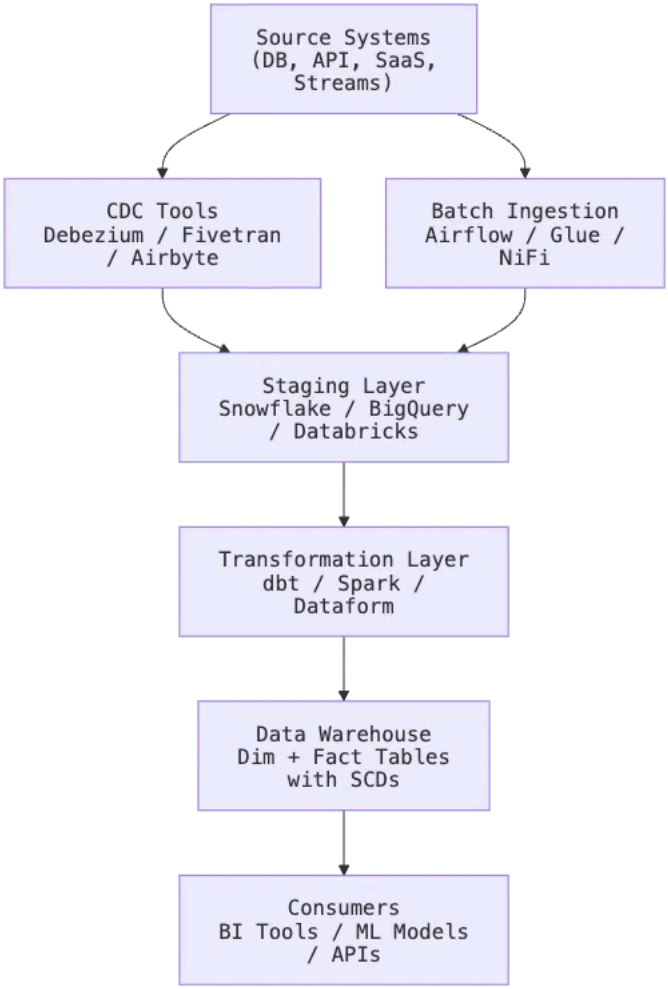
Performance Considerations

Dimension tables with SCD 2 history can grow large. Here's how to keep them performant:

Strategy	Implementation	Benefit
Index natural key + is_current	`CREATE INDEX idx_cust_curr ON dim_customer(customer_id, is_current)`	Fast current-state lookups
Index effective dates	`CREATE INDEX idx_cust_dates ON dim_customer(customer_id, effective_start_date, effective_end_date)`	Fast as-of queries
Partition by load year	`PARTITION BY YEAR(effective_start_date)`	Pruning for historical queries
Materialized view for current state	`CREATE MATERIALIZED VIEW dim_customer_current AS SELECT ... WHERE is_current = TRUE`	Near-instant current-state queries
Archive old versions	Move records with `effective_end_date < (NOW() - INTERVAL '7 years')` to cold storage	Manage table growth

Part 6: Tools & Technologies

The Modern ETL/ELT Stack



Orchestration

Tool	Best For
Apache Airflow	Complex DAG-based orchestration, mature ecosystem
dbt	SQL-first transformation, SCD Type 2 via snapshots
Dagster	Asset-oriented pipelines, strong observability
Prefect	Developer-friendly, dynamic pipelines

Warehouses

Platform	SCD Strengths
Snowflake	Excellent MERGE support, Time Travel for point-in-time, Streams + Tasks for CDC
BigQuery	Partitioned tables, MERGE, time-travel via snapshots
Databricks	Delta Live Tables, Delta Lake MERGE, ACID transactions
Redshift	Good MERGE, materialized views, RA3 node caching

CDC Tools

Tool	Type	Best For
Debezium	Open-source, log-based	Self-hosted, high-volume
Fivetran	Managed	Fast setup, 300+ connectors
Airbyte	Open-source + managed	Flexible, self-hosted option
Striim	Enterprise	Real-time streaming CDC

dbt Snapshots: SCD Type 2 Out of the Box

One of dbt's most powerful features is native SCD 2 support via snapshots:

```
# snapshots/dim_customer_snapshot.sql
{% snapshot dim_customer_snapshot %}
    {{
        config(
          target_schema = 'snapshots',
          unique_key    = 'customer_id',
          strategy      = 'timestamp',
          updated_at    = 'updated_at'
        )
    }}

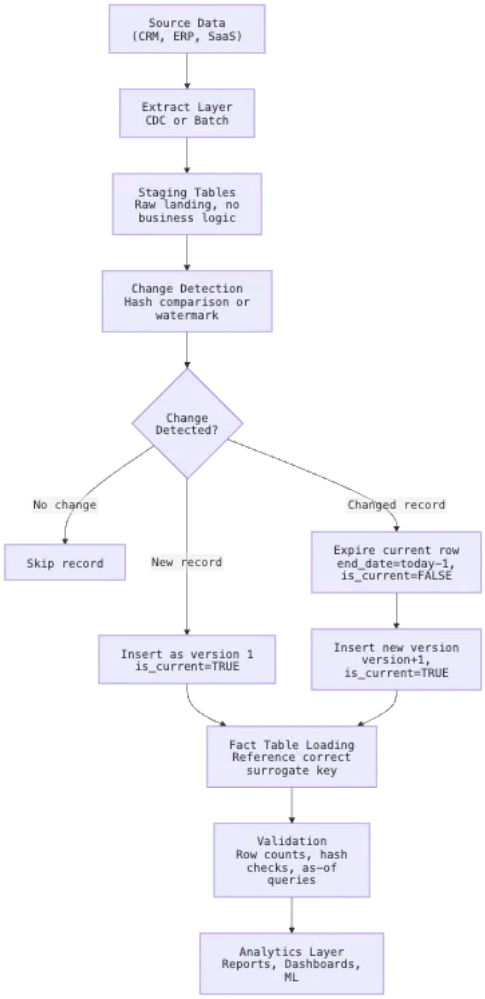
    SELECT * FROM {{ source('raw', 'customer') }}

{% endsnapshot %}
```

dbt handles the SCD 2 logic automatically — inserting new rows, expiring old ones, and maintaining `dbt_valid_from` and `dbt_valid_to` columns. For teams already using dbt, this is often the lowest-friction path to SCD 2.

Putting It All Together: A Reference Architecture

Here's the complete ETL-to-SCD lifecycle in one diagram:



Conclusion: History Is Not Optional

The Monday morning disaster at the start of this guide wasn't caused by bad engineers. It was caused by a silent, dangerous assumption: that data always means current data.

In the real world, facts are tied to moments in time. Customers change cities. Products change categories. Account managers change territories. Sales regions get restructured. If your dimension tables don't track these changes — you're not running a data warehouse. You're running a very expensive snapshot.

Slowly Changing Dimensions are how we preserve truth over time.

Here's the TL;DR for those who need to get back to their pipelines:

- Type 0: Never changes — lock it down
- Type 1: Overwrite — use only for corrections and non-analytical fields
- Type 2: New row for every change — the default choice for analytical dimensions

- Type 3: Previous value in a column — quick but limited
- Type 4: Current table + history table — good for high-query-volume current state
- Type 6: All three combined — maximum flexibility, maximum complexity

When migrating between types:

- Always use shadow tables — never alter in place
- Run dual pipelines during transition
- Use views as your cutover mechanism
- Validate with as-of queries before going live
- Never lose your audit history

Whether you are building your first data warehouse from scratch or evolving a mature platform that's been running for a decade, these patterns will save you from the kind of Monday morning that nobody wants to live through.

Because in data engineering, the best answer to *"what was true on January 15th?"* is never: *"we don't know."*

Data Warehouse + Sql + Big Data + Data Analysis + Data Engineering +



Written by Anurag Sharma

131 followers · 6 following

Edit profile

Data Engineering Specialist with 11+ Exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!

No responses yet



Anurag Sharma him/he

What are your thoughts?